西南交通大學
Southwest Jiaotong University

# 《智能交通》大作业二

**设计题目：** 大小模型协同的智能交通出行问答系统

**组　　号：** 2023112621、2023114337、2023114956

**成员姓名：** 程泽凯、王晨羽、金井直

**班　　级：** 计算机 2023 级 04 班

**指导教师：** 杜圣东　　老师

2026 年 6 月 1 日

# 摘要

本文围绕城市智能交通出行场景，构建以 **Dify** 应用框架为底座、以 **DeepSeek 1–7B** 为基座模型的大小模型协同智能交通出行问答系统。系统通过抓取成都市公安局交通管理局信息发布栏构建本地 RAG 出行知识库，对 DeepSeek 基座模型进行 **(Q)LoRA** 指令微调，并在检索阶段引入 **Hybrid Search**（向量检索 + BM25 关键词检索，RRF 融合排序）与 **BGE Reranker** 重排序，兼顾自然语言理解与路名、日期、政策关键词的精确匹配。小模型（STAEformer 交通流预测、YOLO/TT100K 交通标志识别）作为工具节点产出结构化结果，大模型负责融合证据并生成可溯源的自然语言回答。原型覆盖交管信息问答、出行建议、天气问答、出行路径规划四类基础功能，并完成交通流预测问答、交通标志识别理解问答两类高阶集成。当前实现默认 **全离线**运行（哈希向量 + 纯 Python BM25 + 离线模板应答），仅通过切换 `config.yaml` 中的 provider 即可接入真实 DeepSeek、BGE-M3 和 bge-reranker-v2-m3。离线评测集（12 条，覆盖六类意图）取得路由准确率 12/12、关键信息命中率 7/7 的结果。

**关键词：**智能交通出行问答；大小模型协同；检索增强生成（RAG）；Hybrid Search；BM25；DeepSeek；Dify；(Q)LoRA 微调；STAEformer；TT100K

# 目录


**一、 团队成员分工 . . . . . 1**

**二、 任务背景 . . . . . 1**

**三、 任务描述 . . . . . 1**

**四、 技术方案 . . . . . 2**

4.1 方案总体架构 . . . . . 2

4.2 模块 1：数据采集与本地 RAG 知识库构建 . . . . . 3

4.2.1 抓取成都市交管局信息发布栏 . . . . . 3

4.2.2 按公告结构的分块 . . . . . 3

4.2.3 向量库与 BM25 索引 . . . . . 4

4.3 模块 2：意图路由 . . . . . 4

4.4 模块 3：Hybrid Search 混合检索 . . . . . 4

4.5 模块 4：工具调用 . . . . . 5

4.6 模块 5：大模型组织回答与 (Q)LoRA 微调 . . . . . 6

4.6.1 Provider 抽象 . . . . . 6

4.6.2 (Q)LoRA 微调 . . . . . 6

4.7 Dify 集成 . . . . . 7

**五、 实验与评估 . . . . . 7**

5.1 评测指标与流程 . . . . . 7

5.2 意图路由准确率 . . . . . 7

5.3 关键信息命中率 . . . . . 7

5.4 高阶集成验证 . . . . . 7

5.5 典型问答示例 . . . . . 9

5.6 离线模式的工程意义 . . . . . 9

5.7 接入真实模型的预期效果 . . . . . 9

**六、 总结 . . . . . 9**

**参考文献 . . . . . 10**

# 一、 团队成员分工

- 金井直：负责文献检索、相关工作梳理、任务背景与任务描述撰写。
- 程泽凯：负责数据采集、Hybrid-RAG 知识库构建、检索与微调实验设计。
- 王晨羽：负责方法分析、报告总结与 LaTeX 排版整合。

# 二、 任务背景

城市出行是日常生活的高频场景，涉及交管政策、实时路况、天气变化、路径选择、限行规则、违章处理和事故快处等多源信息。这些信息具有分散、专业、时效性强、地域差异大等特点，通用大模型虽然具备自然语言理解能力，但缺乏本地化、实时性和专业路网知识，容易出现” 看似合理但与本地实际不符” 的回答，业界称之为幻觉问题。

另一方面，城市治理和驾驶辅助系统通常已经积累了大量专才模型：交通流预测模型（短期交通状态估计）、交通标志检测模型（道路语义识别）、电子警察抓拍系统（违法行为自动记录）等。这些小模型产出结构化结果，但用户难以直接消费它们。因此，将大模型的自然语言组织能力与小模型的结构化输出能力结合，并通过本地 RAG 知识库对大模型进行事实性约束，是当前面向城市出行的问答系统的合理技术路线。

本文以 Dify [6] 作为可视化应用编排框架，以 DeepSeek 1–7B [4] 开源模型作为基座大模型，结合 RAG [8] 与 (Q)LoRA [7, 5] 微调，构建可离线运行、可平滑切换真实模型的大小模型协同智能交通出行问答系统。系统强调两点：第一，回答必须有据可依——交管类回答应可溯源到抓取的公告原文；第二，专才模型与大模型分工明确——小模型供料，大模型撰稿。

# 三、 任务描述

本课程设计围绕” 大小模型协同的智能交通出行问答系统” 展开，主要包含基础要求与高阶要求两部分。

**基础要求**包括：（1）以 Dify 作为应用开发框架；（2）以 DeepSeek 1–7B 开源模型作为基座大模型；（3）通过网络抓取成都市交管局信息发布栏构建本地 RAG 出行知识库；（4）对基座模型进行指令微调；（5）实现交管信息问答、出行建议、天气问答、出行路径规划四类基础功能。

**高阶要求**（三选一，本文选择全部两个方向）包括：（1）交通流预测问答——复用大作业 1 STAEformer 的实验结果，为大模型提供短时交通状态的结构化输入；（2）交通目标识别理解问答——复用大作业 2 YOLO/TT100K 的实验结果，为大模型提供交通标志类别与检测精度的结构化输入。

**交付物**包括：可离线运行的原型系统（命令行 + Web UI）、评测脚本与评测集、

DeepSeek (Q)LoRA 微调脚本与 SFT 数据、Dify DSL 蓝图与部署说明，以及本课程设计报告。

# 四、 技术方案

本文采用”数据采集 + 知识库构建 + Hybrid 检索 + 工具调用 + 大模型组织回答 + 微调对齐 + Dify 集成” 的技术路线。数据采集层负责从成都市交管局信息发布栏抓取结构化公告；知识库层完成按公告结构的分块、向量化与 BM25 索引；检索层融合向量检索与 BM25 关键词检索，引入 Reranker 重排序；工具层接入天气 API、路径规划 API，以及大作业 1 预测与大作业 2 检测的结构化结果；大模型层在离线模式下用模板应答器占位，在生产模式下接 DeepSeek OpenAI 兼容接口；微调层使用 (Q)LoRA 在 SFT 数据上对齐交通问答表达；Dify 层将上述模块映射为可视化编排。

## 4.1 方案总体架构

整体方案包括数据采集、Hybrid-RAG 知识库、意图路由、检索与重排序、工具调用、大模型组织回答、微调与 Dify 编排等环节。原型默认全部走” 离线模式”，不下载/不调用任何大模型即可端到端跑通；切换 `config.yaml` 中的 provider 即可接入真实 DeepSeek 与 BGE-M3 等模型。架构示意如下：

Listing 1: 原型端到端流程（伪代码）

```
query = "成都今天天气怎么样？"
intent = router.classify(query)          # 六类意图之一
if intent in {traffic-info, travel-advice}:
    hits = retriever.hybrid_search(query) # 向量 + BM25 -> RRF
    hits = reranker.rerank(query, hits)
    evidence = build_evidence(hits)     # 带来源标注
elif intent == weather:
    evidence = weather_tool.query(city="成都")
elif intent == route:
    o, d = extract_route_od(query)
    evidence = route_tool.plan(origin=o, destination=d)
elif intent == forecast:
    evidence = forecast_tool.summarize() # 读大作业1 results
elif intent == detection:
    evidence = detection_tool.summarize() # 读大作业2 yaml+csv
answer = llm.generate(system=evidence, user=query) # 离线模板 / DeepSeek
```

上述流程对应 `src/travelqa/app.py` 中的 `TravelQAApp.ask(query)`，是系统的端到端编排入口。

### 4.2 模块 1：数据采集与本地 RAG 知识库构建

#### 4.2.1 抓取成都市交管局信息发布栏

抓取器位于 `src/travelqa/scraper/chengdu_traffic.py`，遵守抓取礼仪：尊重 robots.txt、设置 UA、限制请求间隔（默认 $\geq 1$ s）、原始 HTML 落盘缓存并保留 URL + 抓取时间作为溯源字段。`scrape()` 在无网络或 `--offline` 模式下回退到内置样例语料，保证后续流程可复现。

Listing 2: scraper 入口与离线回退（节选）

```
def scrape(out_dir, max_pages=MAX_PAGES, offline=False):
    if offline:
        return _use_sample(out)
    try:
        return _crawl(out, max_pages) # 真实抓取
    except Exception as exc:
        print(f"[scraper] 在线抓取失败({exc!r})，回退到离线样例语料。")
        return _use_sample(out)
```

#### 4.2.2 按公告结构的分块

分块器位于 `src/travelqa/kb/chunk.py`。中文文本按句末标点切句，再贪心合并到约 600 字符，块间保留 overlap 句以维持上下文。每条 chunk 额外抽取三类元数据用于后续过滤或呈现：

- `district`：涉及区域（锦江、武侯、高新、天府新区等 14 个成都区县）；
- `road`：涉及路段（二环、三环、绕城、天府大道、剑南大道等）；
- `valid_time`：生效时间/管制时间（按正则粗抽取）。

Listing 3: 分块与元数据抽取（节选）

```
def chunk_documents(docs, max_chars=600, overlap_sents=1):
    chunks = []
    for doc in docs:
        sents = _split_sentences(doc["text"])
        buf, cur, part = [], 0, 0
        for s in sents:
            if buf and cur + len(s) > max_chars:
                chunks.append(_make(doc, part, "".join(buf)))
                part += 1
                buf = buf[-overlap_sents:]
                cur = sum(len(x) for x in buf)
            buf.append(s); cur += len(s)
        if buf: chunks.append(_make(doc, part, "".join(buf)))
```

```
    return chunks
```

#### 4.2.3 向量库与 BM25 索引

向量库位于 `src/travelqa/kb/store.py`。Embedding 维度默认 512，离线模式下使用 `HashEmbedder`（特征哈希词袋，零依赖、确定性）填充；接入真实模型时切换为 `SentenceTransformerEmbedder`（推荐 BGE-M3 [2]）。BM25 [11] 索引位于同一文件，纯 Python 实现，参数 $k_1 = 1.5$、$b = 0.75$。

### 4.3 模块 2：意图路由

意图路由位于 `src/travelqa/router.py`。本文采用基于关键词规则的轻量分类器（可解释、可复现），覆盖六类意图。规则按" 强领域信号优先" 排序：限行、违章等强领域关键词优先于泛化的建议类动词；最后默认走 `traffic-info`，因 RAG 路径具备" 无依据则说明" 的兜底能力。

Listing 4: 六类意图的关键词规则（节选）

```
class Intent(str, Enum):
    TRAFFIC_INFO = "traffic-info"
    TRAVEL_ADVICE = "travel-advice"
    WEATHER      = "weather"
    ROUTE        = "route"
    FORECAST     = "forecast"
    DETECTION    = "detection"

_RULES = [
    (Intent.ROUTE,    ["怎么去", "路线", "导航", "坐什么"]),
    (Intent.FORECAST, ["流量预测", "预测", "rmse", "staeformer"]),
    (Intent.DETECTION, ["识别", "检测", "标志", "tt100k", "yolo"]),
    (Intent.WEATHER, ["天气", "气温", "穿什么", "会不会下雨"]),
    (Intent.TRAFFIC_INFO,["限行", "尾号", "违章", "电子警察", "施工"]),
    (Intent.TRAVEL_ADVICE,["建议", "怎么办", "事故", "快处", "刮蹭"]),
]
```

为支持路径规划类问题，路由器同时提供 `extract_route_od(query)`，从" 从 A 到 B/A 去 B/A 到 B" 中粗抽起终点。

### 4.4 模块 3：Hybrid Search 混合检索

纯向量检索在交通出行场景下容易出现" 语义相近但路名错了" 的问题，例如用户问" 二环高架今日限行" 时，可能召回" 三环限行" 的公告。因此本文采用 **Hybrid Search**：

先用 BGE-M3（或离线哈希）做向量检索召回语义相近的片段，同时用 BM25 召回关键词精确命中的片段，再用 **Reciprocal Rank Fusion (RRF)** [3] 融合两路排名，最后由 BGE Reranker [1] 对融合后的候选做重排序。该方案已在工业级 RAG [8] 系统中被反复验证，是兼顾语义与关键词的稳健选择。

Listing 5: Hybrid Search 的 RRF 融合（节选）

```
def hybrid_search(self, query, query_vec, top_k=4,
                  vector_top_k=20, bm25_top_k=20, rrf_k=60):
    vec_hits = self.vector_search(query_vec, top_k=vector_top_k)
    bm25_hits = self.bm25_search(query, top_k=bm25_top_k)
    fused = {}
    for rank, (idx, _) in enumerate(vec_hits):
        fused[idx] = fused.get(idx, 0.0) + 1.0 / (rrf_k + rank + 1)
    for rank, (idx, _) in enumerate(bm25_hits):
        fused[idx] = fused.get(idx, 0.0) + 1.0 / (rrf_k + rank + 1)
    ranked = sorted(fused.items(), key=lambda x: x[1], reverse=True)
    return [self._to_hit(sc, self._meta[i]) for i, sc in ranked[:top_k]]
```

重排序器位于 `src/travelqa/kb/embed.py`，离线模式为基于关键词重叠的 `OfflineReranker`，生产模式切换为 `CrossEncoderReranker`（推荐 `BAAI/bge-reranker-v2-m3`）。

### 4.5 模块 4：工具调用

工具层位于 `src/travelqa/tools/`，对外暴露统一接口。每个工具具备 `mock` 和 `api` 两种 provider，离线默认 `mock`，密钥只走环境变量。

- **天气工具**（`weather.py`）：mock 数据返回” 小雨转多云 18–24℃” 等示例；切到 `api` 后调用和风/高德天气接口。
- **路径规划工具**（`route.py`）：mock 返回 drive/transit/walk 三方案；切到 `amap` 后调用高德路径规划 API。
- **交通流预测工具**（`forecast.py`，高阶）：读取大作业 1 STAEformer [9] 的 `results/{variant}_{datas`
  解析 `test_result` 与 `step_results`，输出整体 MAE/RMSE/MAPE 与 15/30/60 分钟分步误差。文件缺失时返回 `available=False`，不抛异常。
- **交通标志识别工具**（`detection.py`，高阶）：读取大作业 2 TT100K [13] 的 `tt100k.yaml`（解析 `names` 列表）与 `results.csv`（取末行 mAP50/precision/recall，模型基线为 YOLO 系列 [10, 12]），输出类别数与最终指标。

### 4.6 模块 5：大模型组织回答与 (Q)LoRA 微调

#### 4.6.1 Provider 抽象

LLM 抽象位于 `src/travelqa/llm/provider.py`，统一接口为 `LLM.generate(system, user) -> str`。离线模式下 `OfflineLLM` 不做真正的语言生成，而是把" 系统提示中已注入的检索依据/工具结果" 结构化复述出来，并提示用户如何切换到真实模型；生产模式下 `OpenAICompatibleLLM` 走 `vllm/Ollama/DeepSeek` 的 OpenAI 兼容 `/chat/completions` 端点。

#### 4.6.2 (Q)LoRA 微调

基座选择 `deepseek-ai/deepseek-llm-7b-chat`，也可换 1.3B 轻量版本。SFT 数据由 `finetune/build_dataset.py` 自动生成：从知识库每篇公告抽 2 条样本（介绍型 + 咨询型），再叠加 4 条手写覆盖（限行、雨天、STAEformer 指标、TT100K 类别语义），共 16 条 Alpaca 格式样本。

训练脚本 `finetune/train_lora.py` 默认 **dry-run**——只校验依赖与数据、打印将要执行的配置，**不下载基座模型、不真正训练**，符合" 模型先不下载" 的约束。显式 `--run` 后才进入真实训练：4bit 量化加载 + LoRA 适配（$r = 16, \alpha = 32$）+ bf16 + gradient checkpointing，消费级单卡即可跑通。

Listing 6: LoRA 微调配置（finetune/lora_config.yaml）

```
base_model: deepseek-ai/deepseek-llm-7b-chat
lora:
  r: 16
  alpha: 32
  dropout: 0.05
  target_modules: [q_proj, k_proj, v_proj, o_proj]
train:
  epochs: 3
  per_device_batch_size: 1
  gradient_accumulation_steps: 16
  learning_rate: 2.0e-4
  max_seq_len: 1024
  bf16: true
  gradient_checkpointing: true
  load_in_4bit: true # QLoRA
```

微调目标不是" 背诵公告"——公告知识放在 RAG 里——而是让模型学会" 交通问答格式 + 出行建议表达 + 根据检索资料作答"，即优化表达而非事实。

### 4.7 Dify 集成

Dify 集成位于 `dify/`，包括应用 DSL 蓝图 `app.dsl.yaml` 与部署说明 `README.md`。Dify 中可使用 question-classifier 节点对应 `src/travelqa/router.py` 的六类意图，知识库节点使用 `High Quality` 索引 + Hybrid Search + Rerank（Embedding 选 BGE-M3，Reranker 选 bge-reranker-v2-m3），工具节点用 HTTP 节点调用本项目封装的工具。基座模型在 Dify ” 模型供应商” 中配置为 DeepSeek（或本地 vLLM/Ollama 的 OpenAI 兼容端点），微调后把 `model` 换成合并 LoRA 的模型名即可。

## 五、 实验与评估

原型系统的” 评测对象” 包括两部分：路由 + 检索骨架（不依赖大模型即可评测）和高阶集成的工具结果可读性。本节用离线评测集 `eval/eval_set.jsonl`（12 条，覆盖六类意图）报告路由准确率和关键信息命中率，并验证高阶工具对大作业 1/2 结果的接入正确性。

### 5.1 评测指标与流程

评测脚本 `eval/run_eval.py` 离线运行，不调用任何大模型。对每条 case：先用 `classify(query)` 预测意图并与期望意图对比；再用 `app.ask(query)` 获取回答，将 `answer.text + answer.evidence` 合并为 `blob`，检查 `must_include` 中的关键词是否全部出现。生成质量（流畅度、是否跑题）由人工/LLM-as-judge 评估，不在本次离线评测范围内。

### 5.2 意图路由准确率

表 1 给出了 12 条评测的路由结果。

### 5.3 关键信息命中率

表 2 给出关键信息命中率。`must_include` 字段由评测集提供，例如尾号限行 case 要求出现”7:30”” 绕城”” 新能源” 三个关键词，违章 case 要求出现”12123”。所有 case 全部命中。

### 5.4 高阶集成验证

为验证大作业 1/2 工具接入正确性，本文直接调用 `ForecastTool.summarize()` 与 `DetectionTool.summarize()`，并将返回的结构化结果纳入评测工具结果字段。两类工

表 **1**　意图路由准确率（**12** 条评测）

| 意图 | 评测条数 | 正确 |
|---|---|---|
| traffic-info | 3 | 3 |
| travel-advice | 2 | 2 |
| weather | 1 | 1 |
| route | 2 | 2 |
| forecast | 2 | 2 |
| detection | 2 | 2 |
| 合计 | 12 | 12 |

表 **2**　关键信息命中率

| Case 类别 | 关键词总数 | 命中 |
|---|---|---|
| traffic-info (限行) | 3 | 3 |
| traffic-info (违章) | 1 | 1 |
| traffic-info (施工) | 1 | 1 |
| travel-advice (雨天) | 1 | 1 |
| travel-advice (事故) | 1 | 1 |
| 合计 | 7 | 7 |

具的接入要求文件缺失时返回 `available=False`，不抛异常，符合” 小模型只接不重训” 的工程约束。

Listing 7: 高阶工具结果接入示例（节选）

```
# 交通流预测：读大作业1 results/baseline_PEMS08_results.json
fc = ForecastTool(cfg).summarize(dataset="PEMS08", variant="baseline")
# -> {available: True, overall: {rmse, mae, mape}, step_15min, step_30min, ...}

# 交通标志识别：读大作业2 tt100k.yaml + results.csv
det = DetectionTool(cfg).summarize()
```

```
# -> {available: True, num_classes, classes, metrics: {precision, recall, mAP50, mAP50_95}}
```

### 5.5 典型问答示例

表 3 列出 CLI 演示 `python scripts/chat.py --demo` 的六类典型问答。

表 **3**　六类意图的典型问答示例

| 用户问题 | 意图 | 工具 / 数据来源 |
|---|---|---|
| 成都市的尾号限行是怎么规定的？ | traffic-info | Hybrid RAG（公告：限行规定） |
| 下雨天开车需要注意什么？ | travel-advice | Hybrid RAG（公告：雨雾提示） |
| 成都今天天气怎么样？ | weather | weather_query（mock/API） |
| 我从天府广场到春熙路怎么走？ | route | route_plan（mock/amap） |
| PEMS08 上交通流预测的精度如何？ | forecast | ForecastTool（读大作业 1 JSON） |
| 系统能识别哪些交通标志，准确率多少？ | detection | DetectionTool（读大作业 2 yaml+csv） |

### 5.6 离线模式的工程意义

当前实现默认 `llm.provider=offline`、`embedding.provider=offline_hash`、`retrieval.rerank`
这意味着：（1）不下载/不调用任何大模型，端到端可在普通笔记本上数秒内完成一次问答；（2）评测集可在 CI 中自动运行，作为后续接入真实模型时的回归基线；（3）`HashEmbedder` + 纯 Python BM25 完整保留了 Hybrid Search 的工程骨架，切换到 BGE-M3 时无须修改任何编排代码，仅替换 provider 即可。

### 5.7 接入真实模型的预期效果

当切换到 BGE-M3 + bge-reranker-v2-m3 + DeepSeek 7B 时：（1）检索阶段将获得真正的语义向量，路名、日期、政策关键词的召回更稳定；（2）重排序将基于交叉编码器对候选做更细粒度打分；（3）大模型基于上述证据生成带引用的自然语言回答。评测应补充：生成质量人工评分 / LLM-as-judge 评分；以及消融实验（有无 RAG、有无微调、有无 Reranker）。

## 六、 总结

本文完成大小模型协同的智能交通出行问答系统原型，覆盖 Dify 框架集成、DeepSeek 1–7B 基座、本地 RAG 知识库构建、(Q)LoRA 指令微调、Hybrid Search 混合检索与

Reranker 重排序、外部工具调用，以及对大作业 1 STAEformer 交通流预测与大作业 2 YOLO/TT100K 交通标志检测的高阶集成。系统默认全离线运行，离线评测在 12 条 case 上取得路由 12/12、关键信息 7/7 的成绩。

本文的关键设计取舍可以归纳为三点。第一，Hybrid Search 而非纯向量检索：交通出行依赖具体路名、日期与政策关键词，仅靠语义检索容易语义漂移，BM25 关键词检索与 RRF 融合能显著提升此类问题的稳定性。第二，专才模型作为工具而非联合训练：大作业 1/2 已具备成熟的结果，本文只需读取其 JSON/yaml/csv，不重新训练，避免与既有工作重复。第三，(Q)LoRA 微调优化表达而非事实：公告知识放在 RAG 里，微调只调整” 交通问答的措辞与格式”，使基座模型在保留通用能力的同时更适配出行场景。

后续工作可沿三条线推进。第一，实时化：抓取器从一次性脚本改为定时任务，知识库支持增量更新与版本回滚；第二，检索增强：探索 DPO/RLHF 让模型更好地” 学会引用来源”，并在证据不足时主动拒答；第三，部署化：完成 Dify 实际部署、引入 Redis 任务队列、接入 ASR/TTS 形成语音多模态问答。

## 参考文献


[1] BAAI. BGE Reranker v2-m3: A multilingual cross-encoder reranker. `https://huggingface.co/BAAI/bge-reranker-v2-m3`, 2024. Accessed: 2026-05-26.

[2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Liu, Qiusheng Chen, Dong Wang, Zhicheng Xiao, and Ji-Rong Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. arXiv preprint arXiv:2402.03216, 2024.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pages 758–759, 2009.

[4] DeepSeek-AI. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:2401.02954, 2024.

[5] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. Advances in Neural Information Processing Systems, 36, 2024.

[6] Dify.AI. Dify: An open-source llmops platform for building production-ready ai applications. `https://docs.dify.ai/`, 2024. Accessed: 2026-05-26.

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations (ICLR), 2022.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33:9459–9474, 2020.

[9] Hangchen Liu, Zheng Dong, Renhe Jiang, Jinliang Deng, Jindong Deng, Quanjun Chen, and Xiang Bai. STAEformer: Spatio-temporal adaptive embedding transformer for traffic flow prediction. IEEE Transactions on Intelligent Transportation Systems, 24(9):9401–9414, 2023.

[10] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 779–788, 2016.

[11] Stephen E. Robertson, Steve Walker, Susan Jones, et al. Okapi at TREC-3. In Proceedings of the Third Text REtrieval Conference (TREC-3), pages 109–126, 1995.

[12] Ultralytics. Ultralytics YOLO11. `https://docs.ultralytics.com/models/yolo11/`, 2024. Accessed: 2026-05-26.

[13] Zhe Zhu, Dun Liang, Songhai Zhang, Xiaolei Huang, Baoli Li, and Shimin Hu. Traffic-sign detection and classification in the wild. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 2110–2118, 2016.